TRANSLATION HIGH-LEVEL AND ASSEMBLY LANGUAGE PROGRAM

CONTENTS

01. Load and store instructions

02. Flow control instructions

03. If-else statements

04. Loops

05. Pseudo-instructions

06. Addressing modes

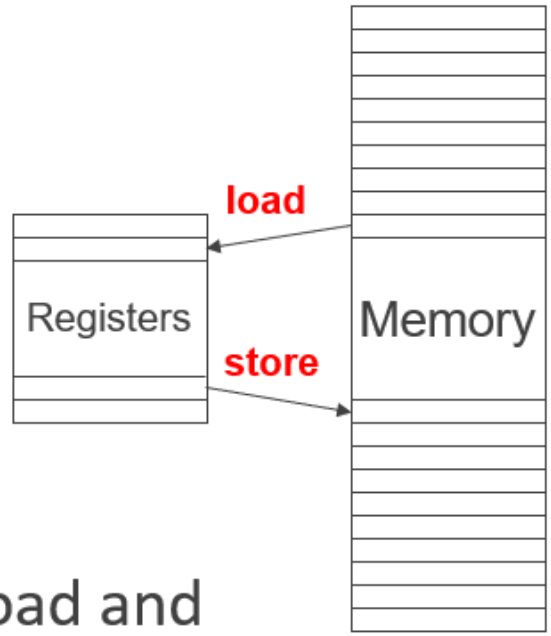
07. Array indexing and traversal

LOAD AND STORE INSTRUCTIONS

Moving forward, let's discuss Load and Store Instructions. These instructions are crucial for accessing and transferring data between memory and registers. We will explore the functionality of load and store instructions, their role in data transfer, and the various addressing modes used to specify operands. Understanding load and store instructions is essential for efficient memory management and data manipulation. Let's proceed!

LOAD AND STORE INSTRUCTIONS

- Instructions that transfer data between memory & registers
- Programs include variables such as arrays and objects
- Such variables are stored in memory
- Load Instruction:
  - Transfers data from memory to a register
- Store Instruction:
  - Transfers data from a register to memory
- Memory address must be specified by load and store

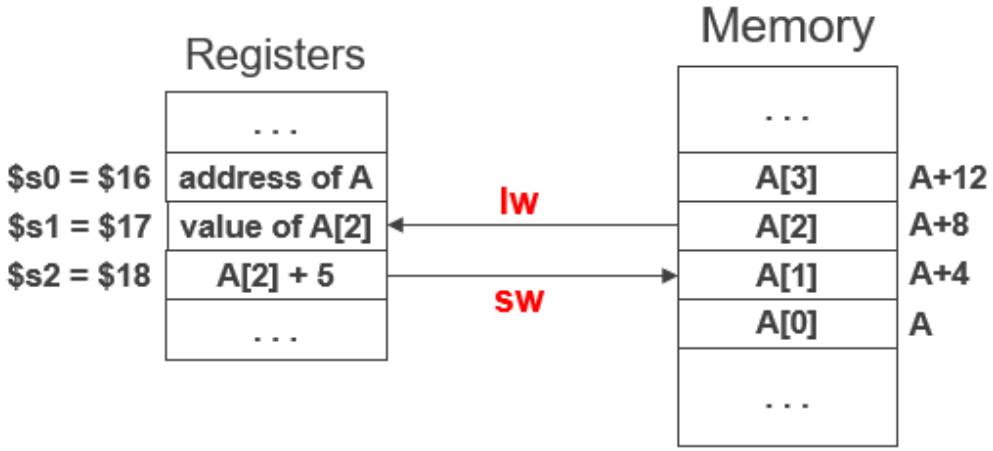


LOAD AND STORE INSTRUCTIONS

- Translate A[1] = A[2] + 5 (A is an array of words)
  - Assume that address of array A is stored in register $s0

```
lw     $s1, 8($s0)     # $s1 = A[2]
addiu  $s2, $s1, 5     # $s2 = A[2] + 5
sw     $s2, 4($s0)     # A[1] = $s2
```

- Index of a[2] and a[1] should be multiplied by 4.



LOAD AND STORE INSTRUCTIONS

Instruction	Meaning	I-Type Format			
lb rt, imm^{16}(rs)	rt = MEM[rs+imm^{16}]	0x20	rs^5	rt^5	imm^{16}
lh rt, imm^{16}(rs)	rt = MEM[rs+imm^{16}]	0x21	rs^5	rt^5	imm^{16}
lw rt, imm^{16}(rs)	rt = MEM[rs+imm^{16}]	0x23	rs^5	rt^5	imm^{16}
lbu rt, imm^{16}(rs)	rt = MEM[rs+imm^{16}]	0x24	rs^5	rt^5	imm^{16}
lhu rt, imm^{16}(rs)	rt = MEM[rs+imm^{16}]	0x25	rs^5	rt^5	imm^{16}
sb rt, imm^{16}(rs)	MEM[rs+imm^{16}] = rt	0x28	rs^5	rt^5	imm^{16}
sh rt, imm^{16}(rs)	MEM[rs+imm^{16}] = rt	0x29	rs^5	rt^5	imm^{16}
sw rt, imm^{16}(rs)	MEM[rs+imm^{16}] = rt	0x2b	rs^5	rt^5	imm^{16}

FLOW CONTROL INSTRUCTIONS

Moving forward, let's discuss Flow Control Instructions like jumps and branches and how they affect program flow.
Let's proceed!

CONDITIONAL BRANCH INSTRUCTIONS

- MIPS compare and branch instructions:

  beq Rs,Rt,label branch to **label** if (**Rs == Rt**)

  bne Rs,Rt,label branch to **label** if (**Rs != Rt**)

- MIPS compare to zero & branch instructions

  Compare to zero is used frequently and implemented efficiently

  bltz Rs,label branch to **label** if (**Rs < 0**)

  bgtz Rs,label branch to **label** if (**Rs > 0**)

  blez Rs,label branch to **label** if (**Rs <= 0**)

  bgez Rs,label branch to **label** if (**Rs >= 0**)

SET ON LESS THAN INSTRUCTIONS

- MIPS also provides set on less than instructions

  `slt rd,rs,rt` if (rs < rt) rd = 1 else rd = 0

  `sltu rd,rs,rt` unsigned <

  `slti rt,rs,im16` if (rs < im^{16}) rt = 1 else rt = 0

  `sltiu rt,rs,im16` unsigned <

- Signed / Unsigned Comparisons

  Can produce different results

  Assume `$s0 = 1` and `$s1 = -1 = 0xffffffff`

  `slt $t0,$s0,$s1` results in `$t0 = 0`

  `stlu $t0,$s0,$s1` results in `$t0 = 1`

JUMP, BRANCH, SLT INSTRUCTIONS

Instruction	Meaning	Format			
j label	jump to label	$op^6 = 2$	imm^{26}		
beq rs, rt, label	branch if (rs == rt)	$op^6 = 4$	rs^5	rt^5	imm^{16}
bne rs, rt, label	branch if (rs != rt)	$op^6 = 5$	rs^5	rt^5	imm^{16}
blez rs, label	branch if (rs<=0)	$op^6 = 6$	rs^5	0	imm^{16}
bgtz rs, label	branch if (rs > 0)	$op^6 = 7$	rs^5	0	imm^{16}
bltz rs, label	branch if (rs < 0)	$op^6 = 1$	rs^5	0	imm^{16}
bgez rs, label	branch if (rs>=0)	$op^6 = 1$	rs^5	1	imm^{16}

Instruction	Meaning	Format					
slt rd, rs, rt	rd=(rs<rt?1:0)	$op^6 = 0$	rs^5	rt^5	rd^5	0	0x2a
sltu rd, rs, rt	rd=(rs<rt?1:0)	$op^6 = 0$	rs^5	rt^5	rd^5	0	0x2b
slti rt, rs, imm^{16}	rt=(rs<imm?1:0)	0xa	rs^5	rt^5	imm^{16}		
sltiu rt, rs, imm^{16}	rt=(rs<imm?1:0)	0xb	rs^5	rt^5	imm^{16}		

IF-ELSE

```
if (condition)
{
    // Code block A
}
else
{
    // Code block B
}
```

IF-ELSE

```
if (x < 10)
{
    // Code block A
    result = 1;
}
else
{
    // Code block B
    result = 0;
}
```

```
1  # Load the value of x into a register ($t0)
2  # instruction here to load x into $t0
3
4  # Compare x with 10 and branch if less than
5  bgt $t0, 10, else_block
6
7  # Code block A
8  li $t1, 1        # result = 1
9  j end            # Jump to the end
10
11 # Code block B (else_block)
12 else_block:
13 li $t1, 0        # result = 0
14
15 # exit the if-else condition
16 exit:
```

L O O P S

```java
for (int i = 0; i < 5; i++)
{
    // Code block A
    result += i;
}
```

```mips
# Initialize the loop control variable (i = 0)
li $t0, 0       # $t0 = i

# Loop label (loop_start)
loop_start:

# Compare i with 5 and branch if greater than or equal
bge $t0, 5, loop_exit

# Code block A
add $t1, $t1, $t0   # result += i

# Increment i
addi $t0, $t0, 1

# Jump back to the loop_start
j loop_start

# Loop exit label
loop_exit:
```

PSEUDO-INSTRUCTIONS

This section will cover pseudo-instructions and how they are used to simplify programming. Let's explore further!

PSEUDO-INSTRUCTIONS

- Pseudo-instructions are instructions that are not part of the official instruction set but are provided by the assembler for convenience. These pseudo-instructions are not executed directly by the hardware; instead, they are translated by the assembler into one or more actual MIPS instructions.
- Pseudo-instructions in MIPS ISA are designed to simplify the assembly language code, making it more readable and reducing the effort required for certain common tasks. They can be particularly useful for programmers who want to write more human-friendly code, but they must be aware that the assembler will ultimately translate these pseudo-instructions into the appropriate standard MIPS instructions.

PSEUDO-INSTRUCTIONS

Basic	Source
lui $1,0x00001122	1: li $t1,0x11223344
ori $9,$1,0x00003344	

Basic	Source
addu $8,$0,$9	1: move $t0,$t1

lui $1,0x00001001	8: la $a0,hellomsg
ori $4,$1,0x00000000	

addiu $2,$0,0x00000...	11: li $v0,5

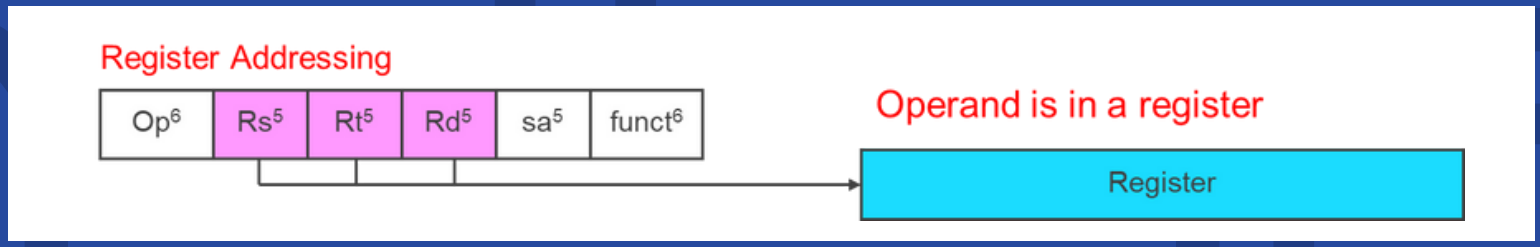
ADDRESSING MODES

Addressing modes determine how operands are accessed in an instruction. There are several addressing modes, including register direct, immediate, indirect, and base-indexed. Register direct mode allows for fast access to data stored in registers. Immediate mode is used for quick calculations and accessing small immediate values. Indirect mode involves accessing the operand through a memory location pointed to by a register, providing flexibility in memory access. Base-indexed mode uses a base register and an index register to calculate the memory address of the operand, commonly used for array accesses and data structures.

ADDRESSING MODES

register

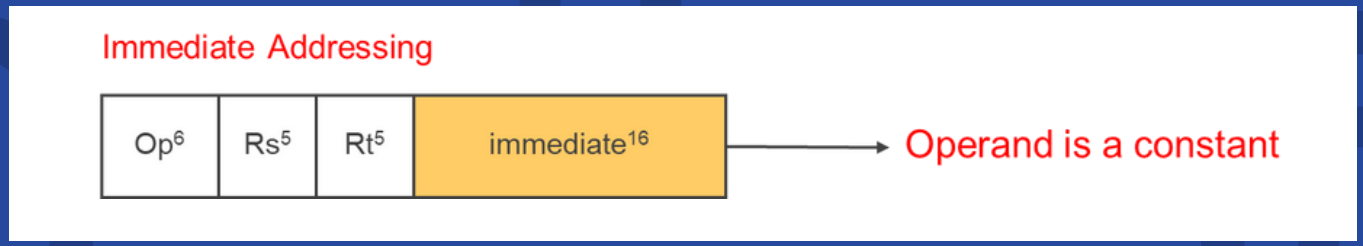
- Register addressing mode uses the value in one or more registers to calculate the memory address.
- Common in R-Type and I-Type instructions.
- Example:
  - lw $t0, ($t1)
    - loads the value at the memory address specified in $t1 into $t0.
    - The memory address is calculated based on the value in $t1.



ADDRESSING MODES

immediate

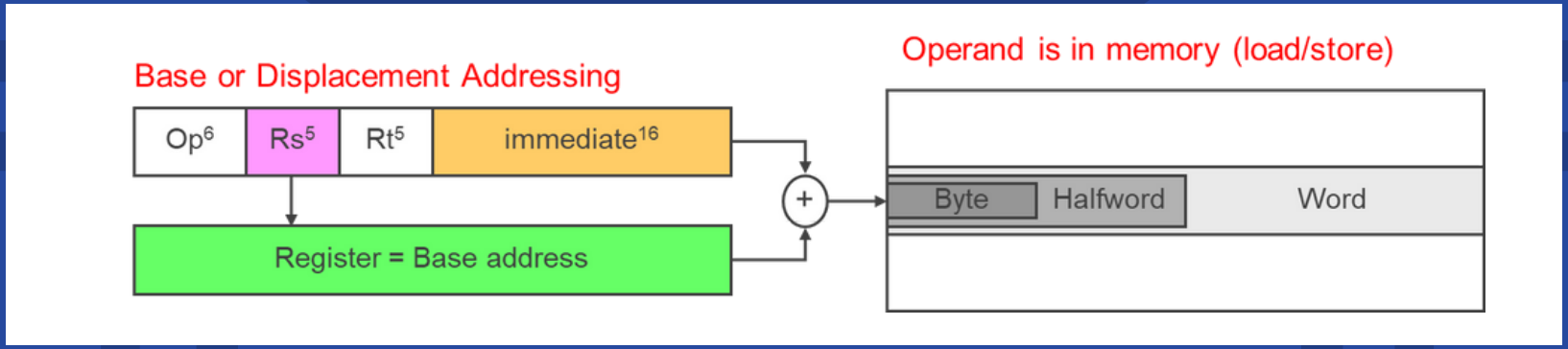
- In immediate addressing mode, the memory address is directly specified within the instruction itself.
- Used in I-Type instructions.
- Example:
  - lw $t0, 100($t1)
    - loads the value at memory address 100 + $t1 into $t0. The immediate value is 100.



ADDRESSING MODES

base/ displacement

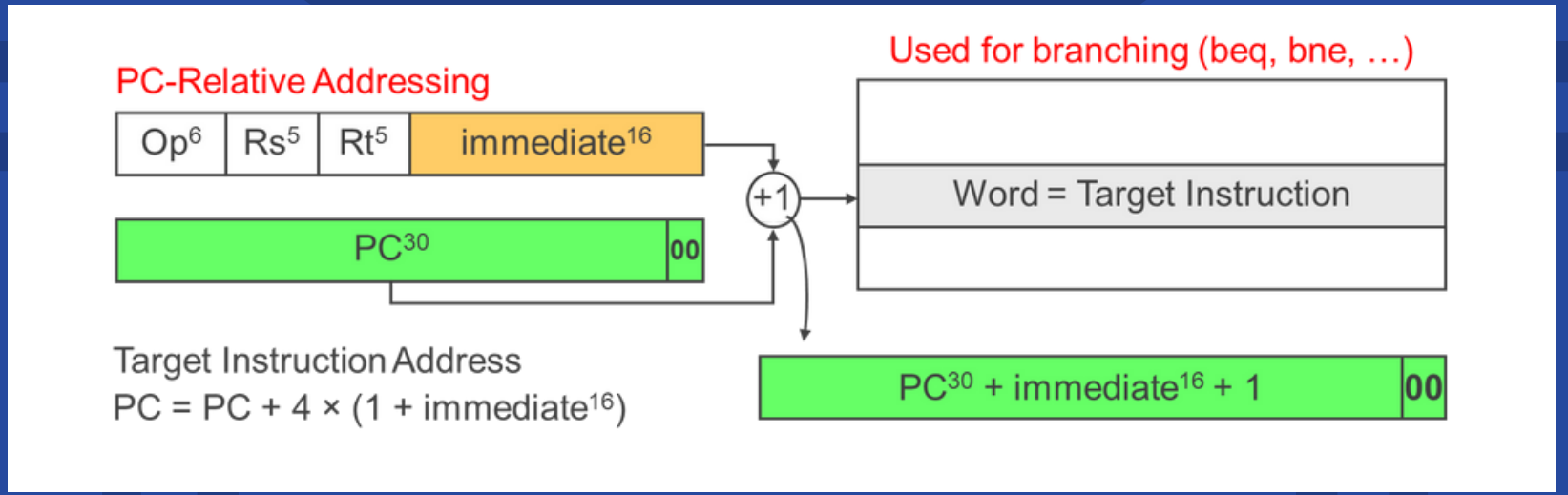
- This addressing mode combines a base address (in a register) with a displacement (immediate value) to calculate the memory address.
- Example:
  - lw $t0, 100($t1)
    - loads the value at memory address 100 + $t1 into $t0.
    - Here, $t1 contains the base address, and 100 is the displacement.



ADDRESSING MODES

PC-relative

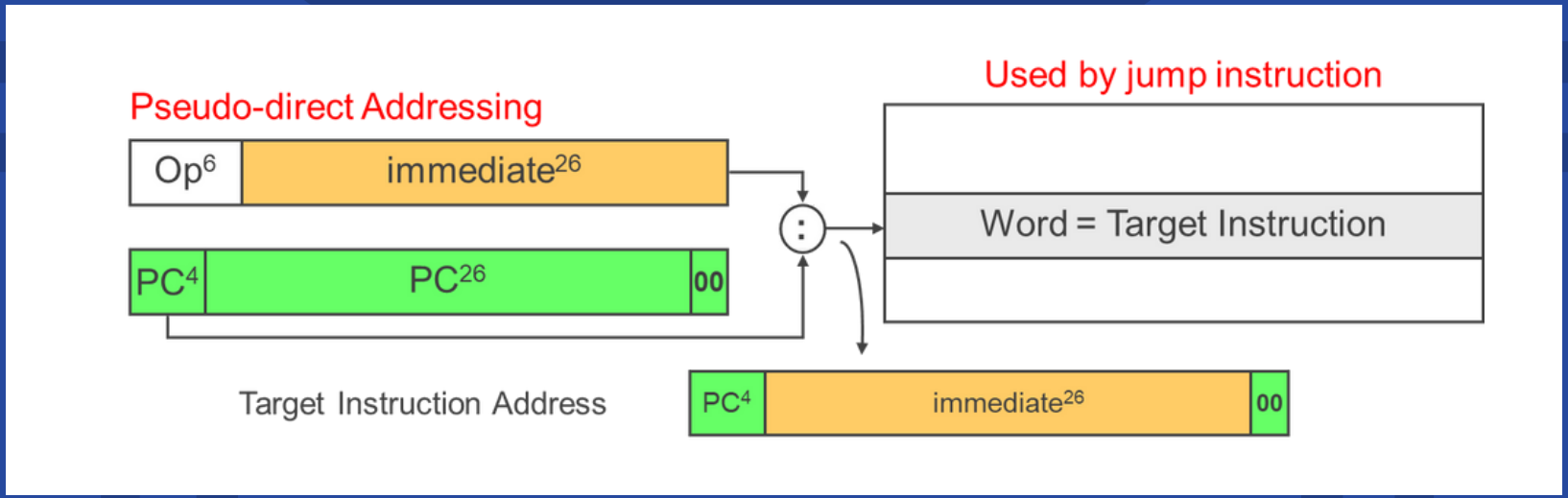
- PC-relative addressing calculates the memory address based on the program counter (PC) value and an offset (immediate value) specified in the instruction.
- Commonly used in branching and jumping instructions.
- Example:
  - beq $t0, $t1, label
    - branches to the label if $t0 is equal to $t1.
    - The branch destination is determined based on the PC value and the offset specified in the instruction.



ADDRESSING MODES

Pseudo-direct

- Pseudodirect addressing is used in MIPS jump instructions, such as j or jal. It calculates the target address using part of the current PC value and a target address specified in the instruction.
- Example:
  - j target jumps to the target address.
  - The target address is calculated by concatenating part of the current PC value with the target address specified in the instruction.



ARRAY INDEXING AND TRAVERSAL

This section will cover array indexing and traversal. Let's explore further!

ARRAY INDEXING AND TRAVERSAL

- Array indexing is the process of accessing specific elements within an array by referring to their position. It's like addressing individual items in a collection, enabling us to perform operations on them selectively.
- Array traversal, on the other hand, involves visiting each element of an array one by one. This is a critical operation when we need to perform tasks like calculating sums, finding specific values, or modifying elements.

ARRAY INDEXING AND TRAVERSAL

```java
int sum = 0;
int myArray[] = {3, 7, 11, 4, 2};
for (int i = 0; i < 5; i++)
{
    sum += myArray[i];
}
```

```
1  .data
2  myArray: .word 3, 7, 11, 4, 2
3  .text
4  main:
5      li $t1, 0                    # Initialize sum to 0
6      li $t2, 0                    # Initialize loop counter to 0
7      loop_start:
8          lw $t0, myArray($t2)     # Load array element into $t0
9          add $t1, $t1, $t0        # Add element to sum
10         addi $t2, $t2, 4         # Move to the next element
11         bne $t2, 20, loop_start # Check loop condition
12     # Result is in $t1
13
```